

SOFTWARE REQUIREMENTS

Unit 1 Chapter 2



A group of people are gathered in a meeting room, looking at a whiteboard. The scene is dimly lit, with the whiteboard being the primary light source. The people are dressed in casual business attire. A woman in the foreground is wearing a denim jacket. A man in the center is wearing a grey sweater. A woman on the right is holding a mug. The whiteboard has some diagrams and text on it, but they are not clearly legible.

Requirements Engineering



What is a Requirement?

A statement that describes what a **system should do** or a **constraint** under which it must operate.

It describes needed services (e.g., generate reports, place orders)

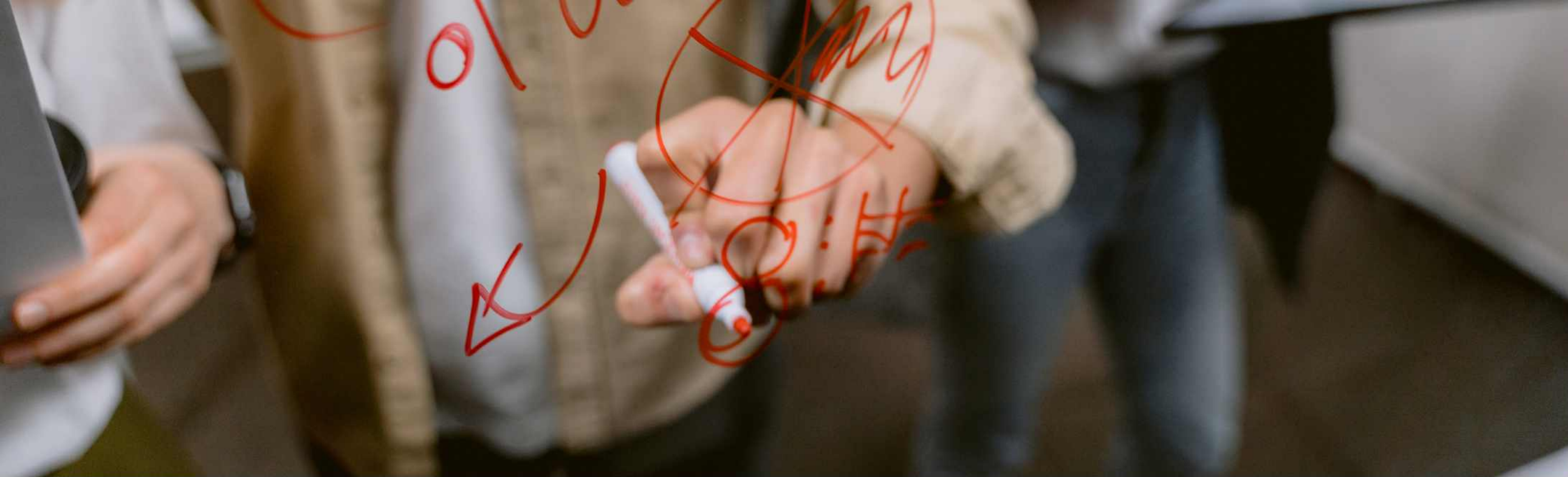
Or constraints (e.g., security, performance, legal rules)

The system shall allow users to create an account.

The system shall allow users to log in using a username and password.

The system must respond to user requests within 2 seconds.

The system shall support at least 1,000 concurrent users.

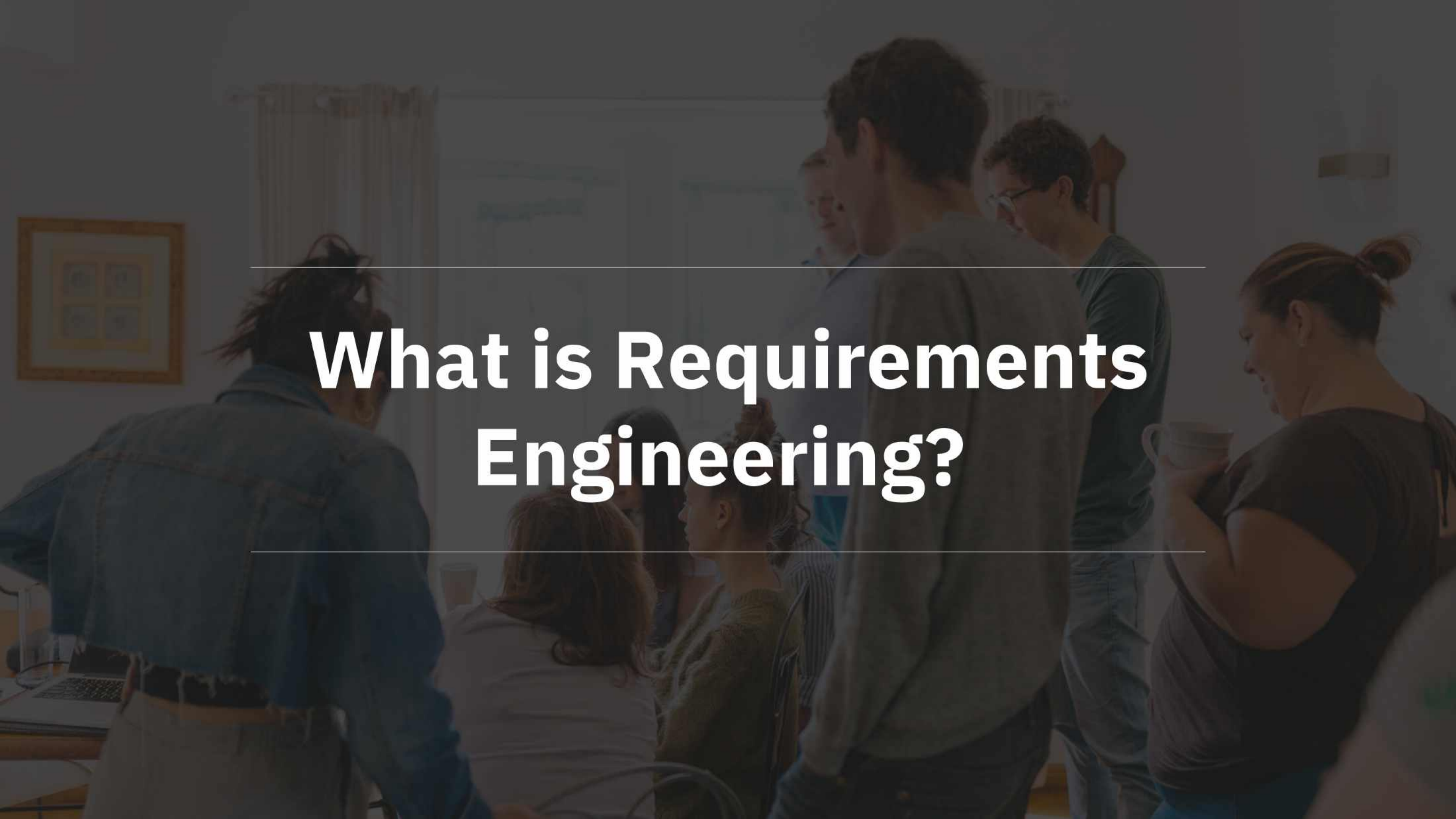


Why Do We Need Requirements?

Capture customer needs, define project scope, act as a contract between client and developer & help avoid misunderstandings, delays, and rework

Without clear requirements:

- Different people imagine different systems
- Projects go over budget and schedule
- Delivered software may be useless to users

A group of people are gathered in a meeting room, looking at a whiteboard. The scene is dimly lit, with the whiteboard being the primary light source. The people are dressed in casual business attire. A woman in the foreground is holding a mug. The overall atmosphere is professional and collaborative.

What is Requirements Engineering?



What is Requirements Engineering?

Requirements Engineering is the process of:

- Finding out requirements (elicitation)
- Analyzing them
- Documenting them
- Checking/validating them



The Problem with the Word “Requirement”

The term requirement is used
inconsistently in industry.



A high-level goal

“The system should generate monthly reports.”



A very detailed specification

“The system shall generate a report at 17:30 on the last
working day of the month.”

- **Before a contract → requirements must be abstract**
- **After a contract → requirements must be detailed and precise**
- **Both documents are often called “requirements”**

Levels of Requirements

To avoid confusion, we separate requirements into



User Requirements

Statements in natural language (may include diagrams) of the services the system provides and its operational constraints. Written for customers.



System Requirements

A structured document setting out detailed descriptions of the system's functions, services and operational constraints. Defines what should be implemented so may be part of a contract between client and contractor.



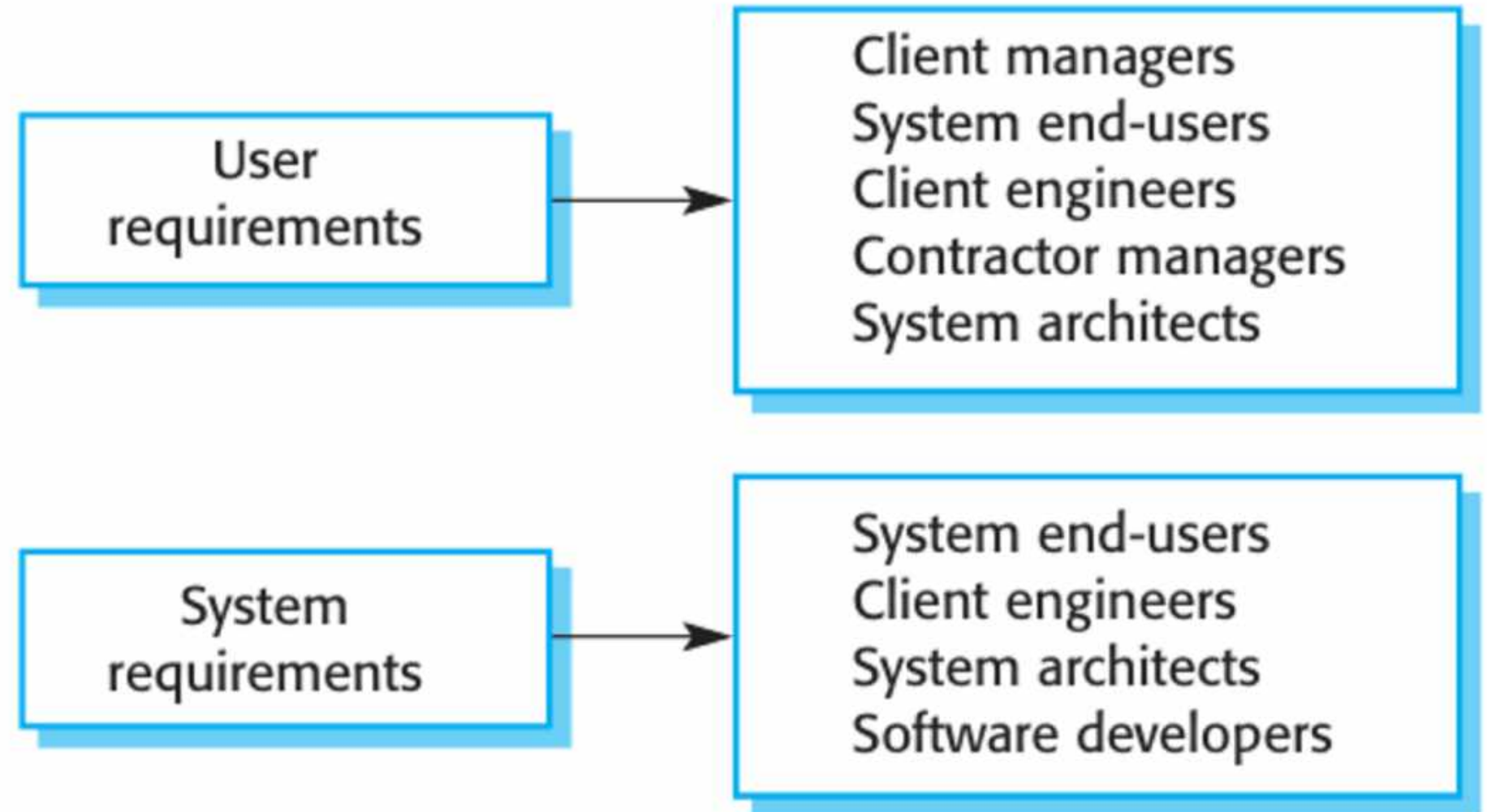
User requirements definition

- 1.** The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System requirements specification

- 1.1** On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
- 1.2** The system shall generate the report for printing after 17.30 on the last working day of the month.
- 1.3** A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
- 1.4** If drugs are available in different dose units (e.g. 10mg, 20mg, etc.) separate reports shall be created for each dose unit.
- 1.5** Access to drug cost reports shall be restricted to authorized users as listed on a management access control list.

Readers of different types of requirements specification





Functional and Non-Functional Requirements

Types of Requirements

Software system requirements are often classified as functional or non-functional requirements



Functional Requirements

These are statements of **services the system should provide**, how the system should react to particular inputs, and how the system should behave in particular situations. In some cases, the functional requirements may also explicitly state what the system should not do.



Non-Functional Requirements

These are **constraints on the services or functions** offered by the system. They include timing constraints, constraints on the development process, and constraints imposed by standards.

Non-functional requirements often apply to the system as a whole rather than individual system features or services.

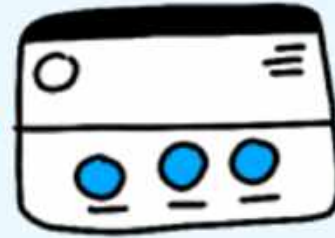


FUNCTIONAL REQUIREMENTS

examples of

NON-FUNCTIONAL REQUIREMENTS

the website will have a homepage



the homepage should load within 1.5 seconds

the website will store customer data



customer data will be encrypted to level 4 encryption standards

website customers can log into their accounts



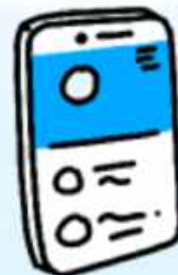
the website has the capacity for 5.000 logged-in users at any one time

the website will always be available to customers



the website must have a 99.7% uptime

customers can access the websites on their phones



the website will be compatible with iOS 12.8 and above

Functional Requirements

- 1 ● The functional requirements for a system describe what the system should do.
- 2 ● These requirements depend on the type of software being developed, the expected users of the software, and the general approach taken by the organization when writing requirements.
- 3 ● When expressed as user requirements, functional requirements should be written in natural language so that system users and managers can understand them.
- 4 ● Functional system requirements expand the user requirements and are written for system developers. They should describe the system functions, their inputs and outputs, and exceptions in detail.

Mentcare: A Patient Information System for Mental Health Care



Example

Mentcare System, used to maintain information about patients receiving treatment for mental health problems

1. A user shall be able to search the appointments lists for all clinics.
2. The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.
3. Each staff member using the system shall be uniquely identified by his or her eight-digit employee number

Problems arise when requirements are not precisely stated.

Requirements Imprecision

Ambiguous requirements may be interpreted in different ways by developers and users.

What does “search” mean?

- By patient name? Date? Appointment type? Doctor? Clinic?
- Developers might implement a simple text search by patient name only.
- Users might expect advanced filters (date, clinic, doctor, status).
- Management might expect access restrictions per clinic.

1. A user shall be able to search the appointments lists for all clinics.
2. The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.
3. Each staff member using the system shall be uniquely identified by his or her eight-digit employee number

Requirements completeness and consistency

In principle, requirements should be both complete and consistent



Complete

They should include descriptions of all facilities required

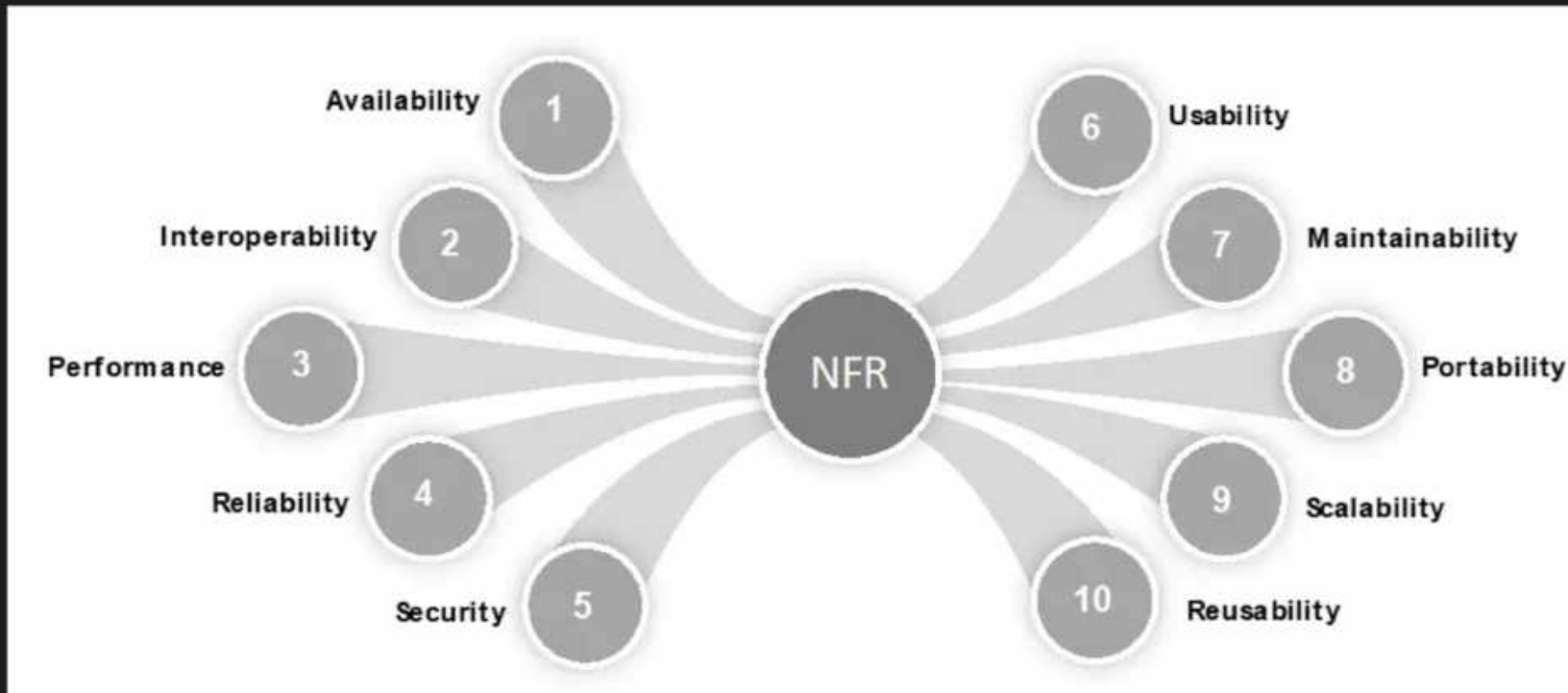


Consistent

There should be no conflicts or contradictions in the descriptions of the system facilities

In practice, it is impossible to produce a complete and consistent requirements document

Non-Functional Requirements



1

These define system properties and constraints e.g. reliability, response time, and storage requirements. Constraints are I/O device capability, system representations, etc

2

Process requirements may also be specified mandating a particular IDE, programming language or development method.

3

Non-functional requirements may be more critical than functional requirements. If these are not met, the system may be useless

4

These non-functional requirements usually specify or constrain characteristics of the system as a whole.

Non-functional Requirements Implementation



Affects the overall system architecture

Some non-functional requirements cannot be satisfied by changing a single module. Instead, they require architectural decisions that influence how the whole system is structured and how components communicate.

Requirement:

“The system shall respond to user requests within 2 seconds.”

Impact on implementation:

- The system may need:
 - Fewer layers of communication between components
 - Local caching of frequently used data
 - Asynchronous processing
 - Data may be stored closer to where it is used

■ Non-functional Requirements Implementation



Generate multiple functional requirements

A single non-functional requirement often leads to:

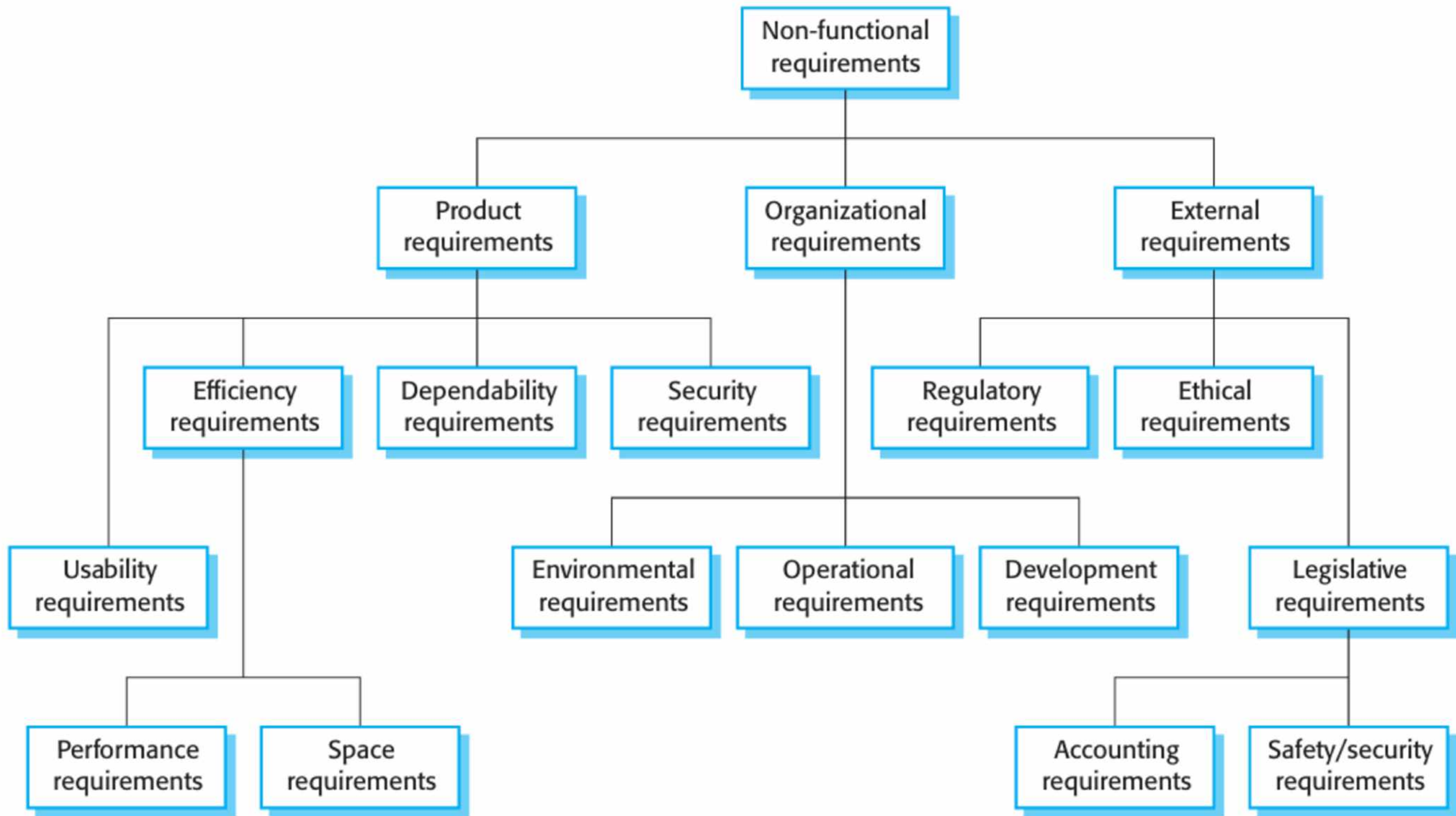
- New functional requirements (new system services)
- Constraints on existing functional requirements

Non-functional requirement:

“Only authorized staff shall be able to access patient medical records.”

Generated functional requirements:

- The system shall provide a user authentication service.
- The system shall allow users to log in and log out.
- The system shall verify user identity using an employee number and password.



Non-functional Classifications



Product requirements

Requirements which specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.



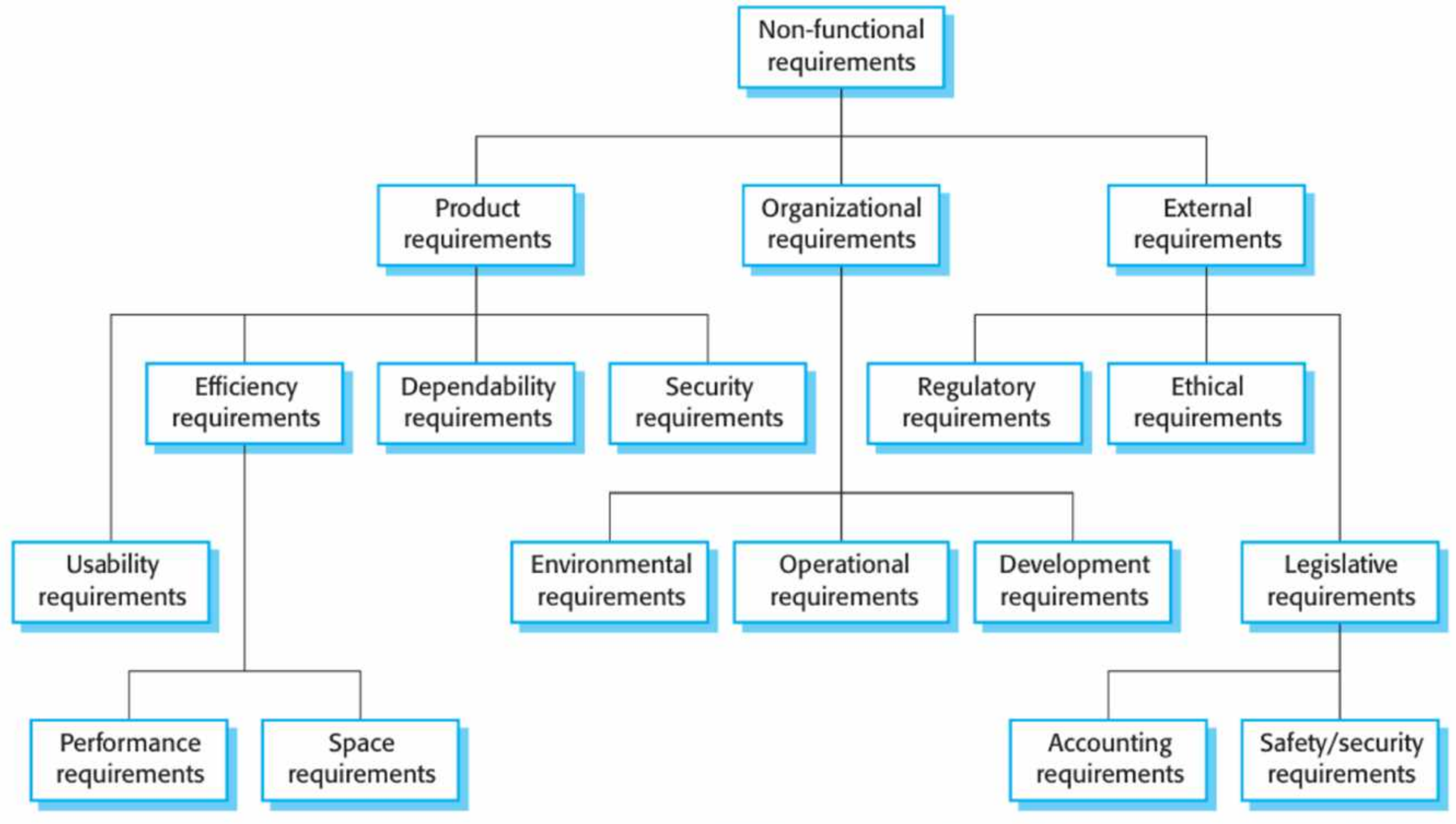
Organizational requirements

Requirements which are a consequence of organizational policies and procedures e.g. process standards used, implementation requirements, etc.



External requirements

Requirements which arise from factors which are external to the system and its development process, e.g. interoperability requirements, legislative requirements, etc



“The system shall be easy to learn for new staff.”

Search results shall be displayed within 2 seconds.

The system shall not use more than 2GB of memory.

“The system shall be available 99.9% of the time.”

“Only authorized staff shall access patient records.”

“The system shall run on Linux servers.”

“System backups shall be performed daily.”

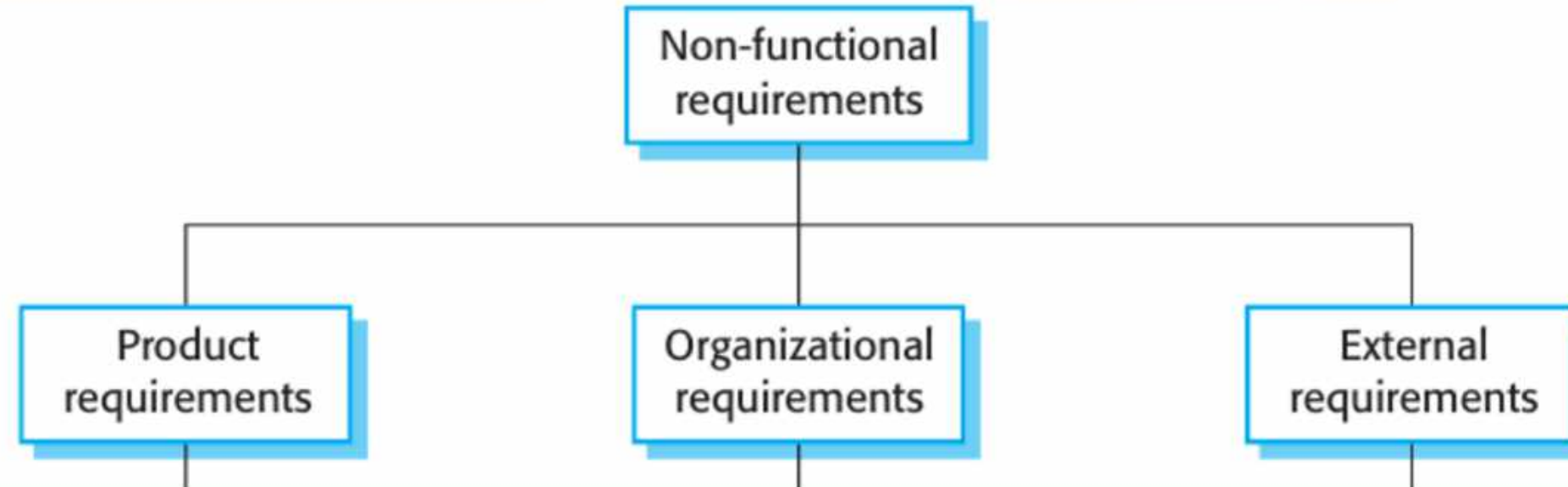
“The system shall be developed using Java and Oracle DB.”

“The system shall comply with healthcare data standards.”

“Patient data shall not be used without consent.”

“All transactions shall be auditable for 7 years.”

“The system shall prevent data breaches.”



Product requirement

The MHC-PMS shall be available to all clinics during normal working hours (Mon–Fri, 0830–17.30). Downtime within normal working hours shall not exceed five seconds in any one day.

Organizational requirement

Users of the MHC-PMS system shall authenticate themselves using their health authority identity card.

External requirement

The system shall implement patient privacy provisions as set out in HStan-03-2006-priv.

Goals

Stakeholders often express **non-functional requirements as general goals:**

“The system should be easy to use and minimize user errors.”

- “Easy to use” is subjective
- Different stakeholders interpret it differently
- Impossible to objectively verify after delivery

This leads to disputes between customers and developers.

Testable non-functional requirements

To solve this, goals should be rewritten as **quantitative, testable non-functional requirements.**

“Medical staff shall be able to use all system functions after two hours of training. After training, the average number of user errors shall not exceed two per hour.”

- Training time can be measured
 - Errors can be counted during testing
 - Objective pass/fail decision is possible
- Developers can design and test the system against clear criteria.

A group of people are gathered in a meeting room, looking at a whiteboard. The room has a whiteboard with diagrams, a framed picture on the wall, and a laptop on a table. The people are dressed in casual business attire. The text "Domain requirements" is overlaid in the center of the image.

Domain requirements

What are domain requirements?

Domain requirements are requirements that come from the operational environment (domain) in which the system will be used.

They are not invented by stakeholders — they already exist because of physical laws, regulations, or established practices in that domain.



Domain knowledge includes:

- Braking distances
- Signal visibility
- Speed limits
- Weather effects on tracks

Software example:

A train control system must:

- Reduce speed in fog or rain
- Increase braking distance in snow
- Stop at red signals automatically

These are physical and safety constraints of the transport domain.



How domain requirements affect the system?



New functional requirements

They may require the system to perform new functions
“The system shall calculate braking distance based on weather conditions.”



Constraints on existing requirements

They may restrict how existing functions work.
“The train shall not exceed 40 km/h on icy tracks.”



Specific computations or algorithms

They may require domain-specific calculations.
“Braking distance shall be calculated using the standard railway braking formula.”

Domain requirements problems

Understandability problem:

Domain requirements are usually written using domain-specific terminology, which software engineers may not fully understand.

A medical system requirement states:

“The system shall flag abnormal systolic readings.”

A developer unfamiliar with medicine may not know:

- What “systolic” means
- What values count as “abnormal”

Result: incorrect implementation.



Domain requirements problems

Implicitness problem

Domain experts often assume certain knowledge is obvious, so they do not explicitly state it in the requirements.

Example:

A railway expert may assume:

- Trains take longer to stop in wet conditions
- Signals must be visible from a minimum distance

Because this is “common knowledge” in the domain, it may never be written down.

Software engineers may:

- Miss the requirement entirely
- Build a system that works in theory but fails in practice



Domain requirements problems

Implicitness problem

Domain experts often assume certain knowledge is obvious, so they do not explicitly state it in the requirements.

Example:

A railway expert may assume:

- Trains take longer to stop in wet conditions
- Signals must be visible from a minimum distance

Because this is “common knowledge” in the domain, it may never be written down.

Software engineers may:

- Miss the requirement entirely
- Build a system that works in theory but fails in practice





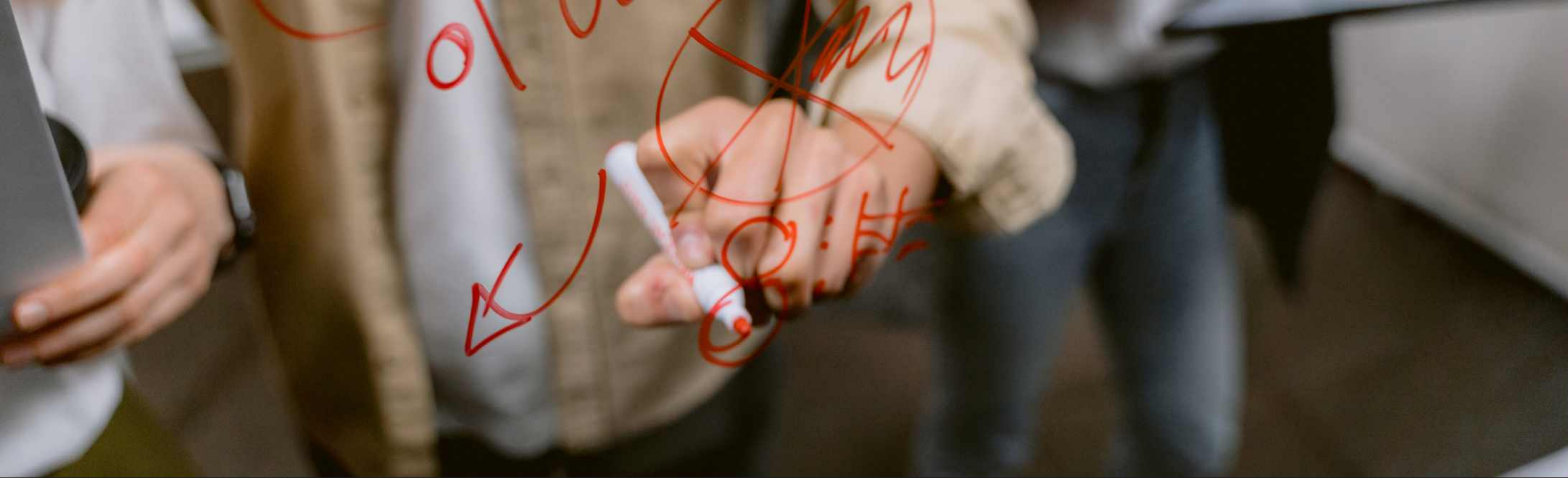
Software Requirements Document (SRS)



The Software Requirements Document, also called the Software Requirements Specification (SRS), is a formal, official description of what is required of the system developers

Can include both a definition of user requirements and a specification of the system requirements.

It is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it.



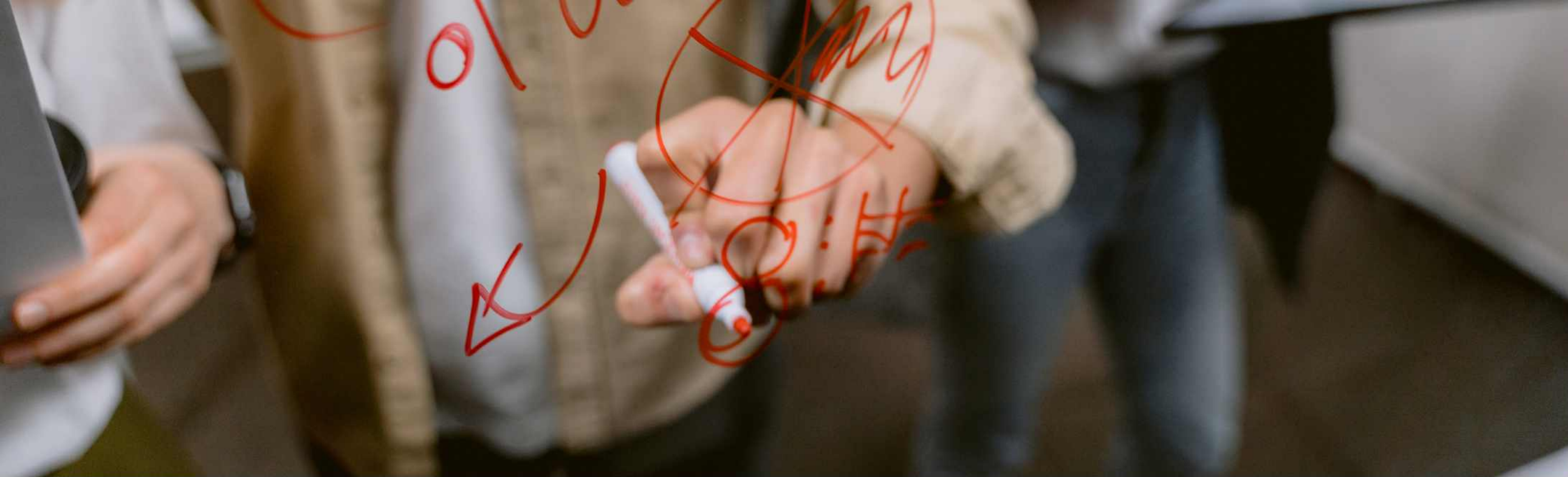
**When is an SRS
essential?**

Outsourced development

- Developers and customers are separate organizations
- SRS becomes a legal reference

Example:

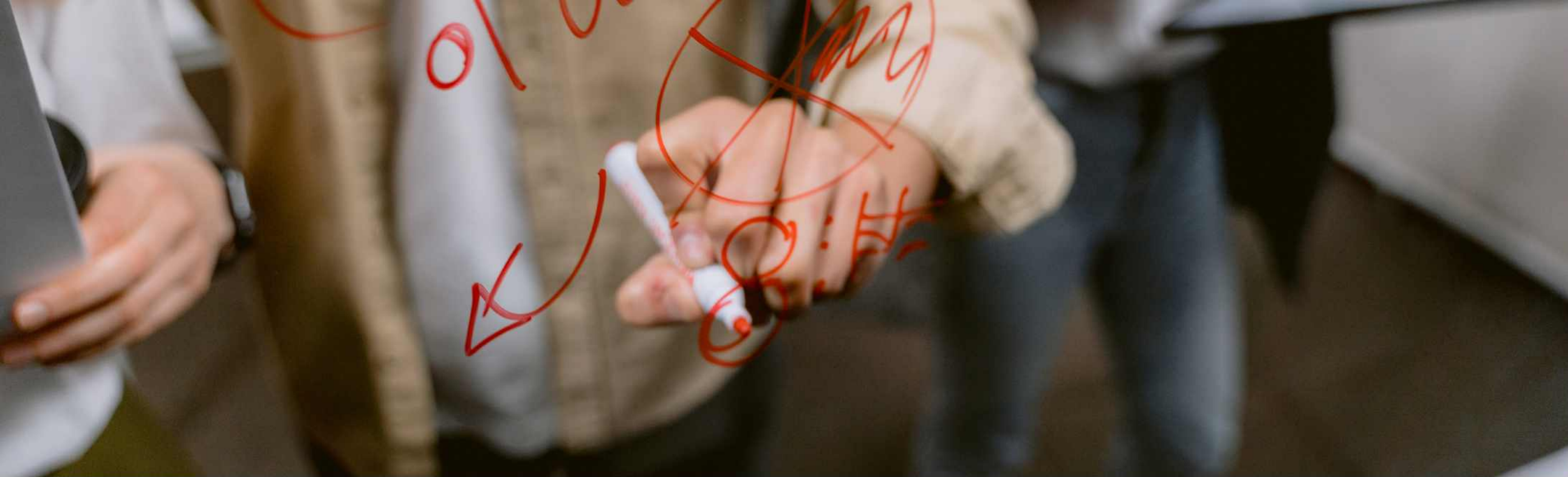
A company hires an external vendor to build payroll software.



**When is an SRS
essential?**

Large systems with multiple teams

- Different teams build different components
- SRS ensures consistency



**When is an SRS
essential?**

Safety-critical or regulated systems

- Formal analysis and verification required
- Example:
Medical devices, aviation systems, banking systems.

Agile view of requirements documentation

Agile methods argue that:

- Requirements change constantly
- Writing detailed documents upfront is often wasted effort

Agile approach:

- Capture requirements as user stories
- Write them on cards or digital boards
- Prioritize stories for each iteration (sprint)



Users of the requirements document

Validate that needs are met

Decide cost, scope, and feasibility

Understand what to build

Create test cases

Understand system behavior

System customers

Specify the requirements and read them to check that they meet their needs. Customers specify changes to the requirements.

Managers

Use the requirements document to plan a bid for the system and to plan the system development process.

System engineers

Use the requirements to understand what system is to be developed.

System test engineers

Use the requirements to develop validation tests for the system.

System maintenance engineers

Use the requirements to understand the system and the relationships between its parts.

A group of people are gathered in a meeting room, looking at a whiteboard. The scene is dimly lit, with the whiteboard being the primary light source. The people are dressed in casual business attire. The whiteboard contains some faint, illegible text. The overall atmosphere is professional and collaborative.

The structure of a requirements document

The structure of a requirements document

Chapter	Description
Preface	This defines the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	This describes the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.
Glossary	This defines the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Here, you describe the services provided for the user. The nonfunctional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System architecture	This chapter presents a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.

The structure of a requirements document

System requirements specification	This describes the functional and nonfunctional requirements in more detail. If necessary, further detail may also be added to the nonfunctional requirements. Interfaces to other systems may be defined.
System models	This chapter includes graphical system models showing the relationships between the system components and the system and its environment. Examples of possible models are object models, data-flow models, or semantic data models.
System evolution	This describes the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system.
Appendices	These provide detailed, specific information that is related to the application being developed—for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.

A group of people are gathered in a meeting room, looking at a whiteboard. The room has a white wall with a framed picture on the left and a window with curtains in the background. The people are dressed in casual business attire. The text "Requirements Specification" is overlaid in the center of the image.

Requirements Specification



Requirement Specification

Requirements specification is the process of writing down the user and system requirements in a requirements document. Ideally, the user and system requirements should be clear, unambiguous, easy to understand, complete, and consistent.

Ways of writing a system requirements specification

Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract

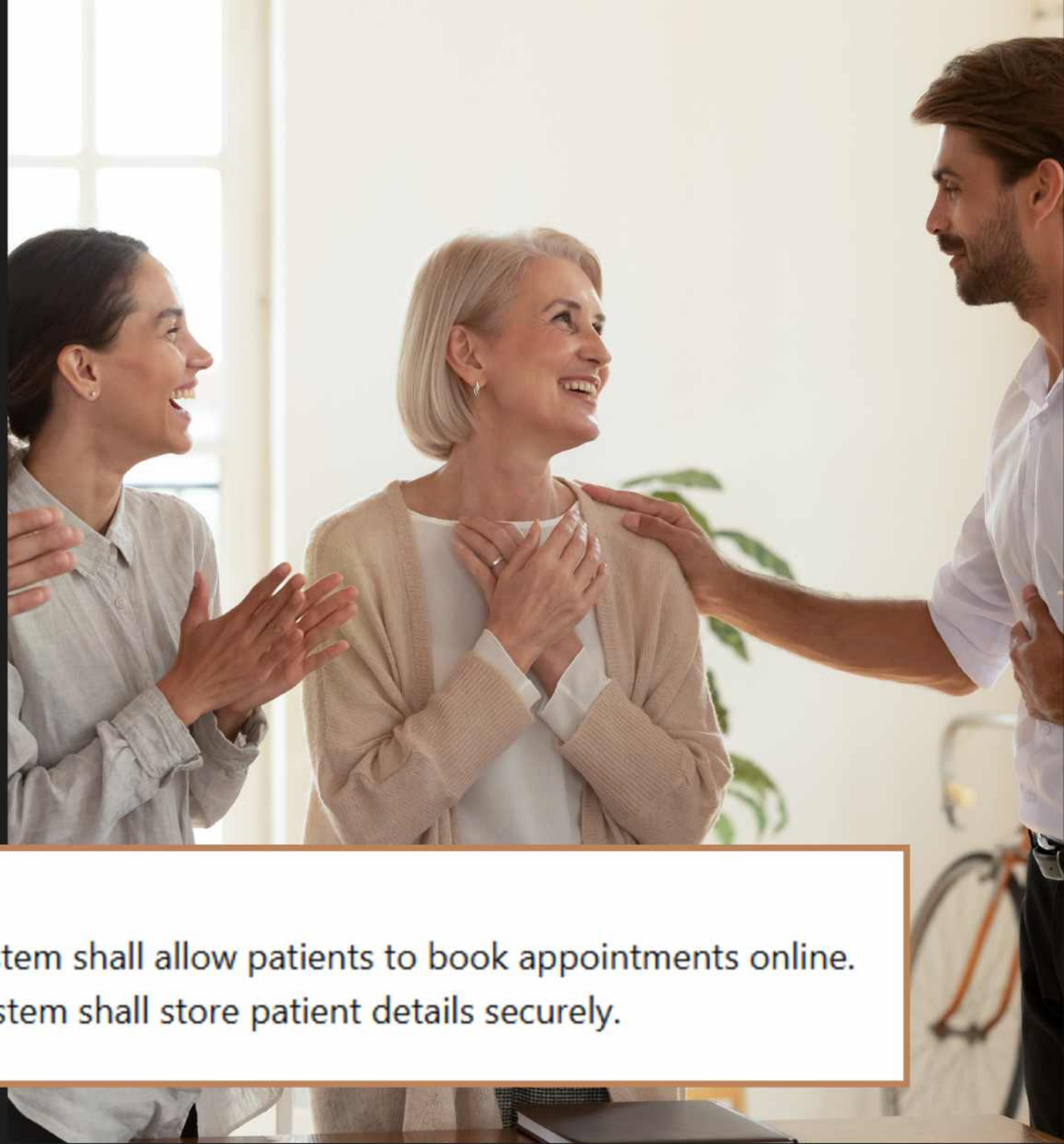
Natural Language

The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.

Example:

R1: The system shall allow patients to book appointments online.

R2: The system shall store patient details securely.



Natural Language Guidelines

Requirement ID

REQ-1.1

Requirement Statement

The system **shall** *allow users to reset their password using their registered email address.*

Rationale

Users frequently forget passwords, and without a reset option they cannot access the system. This reduces support requests and user frustration.

Source

Customer Support Team

- Use a standard requirement format for all requirements. Each requirement should be written in one or two clear sentences of natural language to reduce omissions and simplify checking.

- Use consistent wording to show priority: mandatory requirements shall be written using “shall”, while optional or desirable requirements should use “should.”

- Highlight key parts of each requirement using bold, italics, or color to improve readability and emphasis.

- Avoid technical jargon and acronyms. Write requirements so they can be understood by readers without software or engineering knowledge.

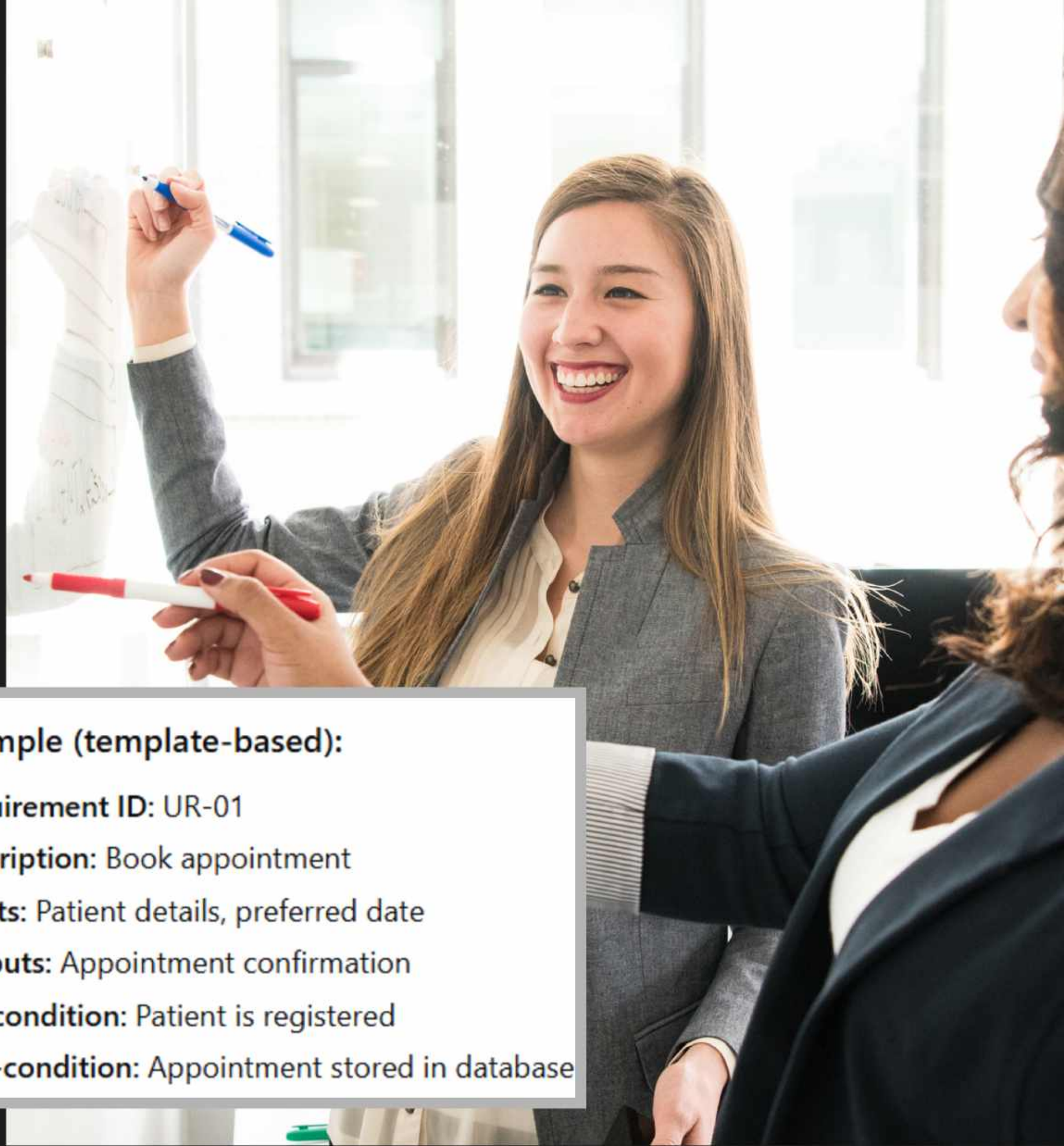
- Include a rationale and source for each requirement whenever possible, explaining why it exists and who requested it, to support future changes and decision-making.

3.2 The system shall measure the blood sugar and deliver insulin, if required, every 10 minutes. (*Changes in blood sugar are relatively slow, so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.*)

3.6 The system shall run a self-test routine every minute with the conditions to be tested and the associated actions defined in Table 1. (*A self-test routine can discover hardware and software problems and alert the user to the fact the normal operation may be impossible.*)

Structured Natural Language

The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.



Example (template-based):

Requirement ID: UR-01

Description: Book appointment

Inputs: Patient details, preferred date

Outputs: Appointment confirmation

Pre-condition: Patient is registered

Post-condition: Appointment stored in database

Structured Natural Language Guidelines

Insulin Pump/Control Software/SRS/3.3.2

Function	Compute insulin dose: Safe sugar level.
Description	Computes the dose of insulin to be delivered when the current r is in the safe zone between 3 and 7 units.
Inputs	Current sugar reading (r2), the previous two readings (r0 and r1)
Source	Current sugar reading from sensor. Other readings from memory
Outputs	CompDose—the dose in insulin to be delivered.
Destination	Main control loop.
Action:	CompDose is zero if the sugar level is stable or falling or if the level increase is decreasing. If the level is increasing and the rate of increase is decreasing, CompDose is computed by dividing the difference between the current level and the previous level by 4 and rounding the result. If the result is rounded down, it is set to the minimum dose that can be delivered. (see Figure 4.14)
Requires	Two previous readings so that the rate of change of sugar level can be computed.
Precondition	The insulin reservoir contains at least the maximum allowed single dose.
Postcondition	r0 is replaced by r1 then r1 is replaced by r2.
Side effects	None.

Use a structured template to specify system requirements, written as standardized forms

A structured functional requirement should include:

1. A brief description of the function or entity.
2. Its inputs and where they come from.
3. Its outputs and where they go.
4. Any required data or dependent system elements.
5. The action performed by the system.
6. Preconditions (what must be true before execution) and postconditions (what is true after execution).
7. Any side effects of the operation.

Structured specifications reduce ambiguity, improve consistency, and make requirements easier to organize.

Insulin Pump/Control Software/SRS/3.3.2

Function	Compute insulin dose: Safe sugar level.
Description	Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.
Inputs	Current sugar reading (r2), the previous two readings (r0 and r1).
Source	Current sugar reading from sensor. Other readings from memory.
Outputs	CompDose—the dose in insulin to be delivered.
Destination	Main control loop.
Action:	CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered. (see Figure 4.14)
Requires	Two previous readings so that the rate of change of sugar level can be computed.
Precondition	The insulin reservoir contains at least the maximum allowed single dose of insulin.
Postcondition	r0 is replaced by r1 then r1 is replaced by r2.
Side effects	None.

Tabular Specification

To reduce ambiguity in natural-language requirements, additional representations such as tables or diagrams can be used. Tables are particularly useful when there are a number of possible alternative situations and you need to describe the actions to be taken for each of these.

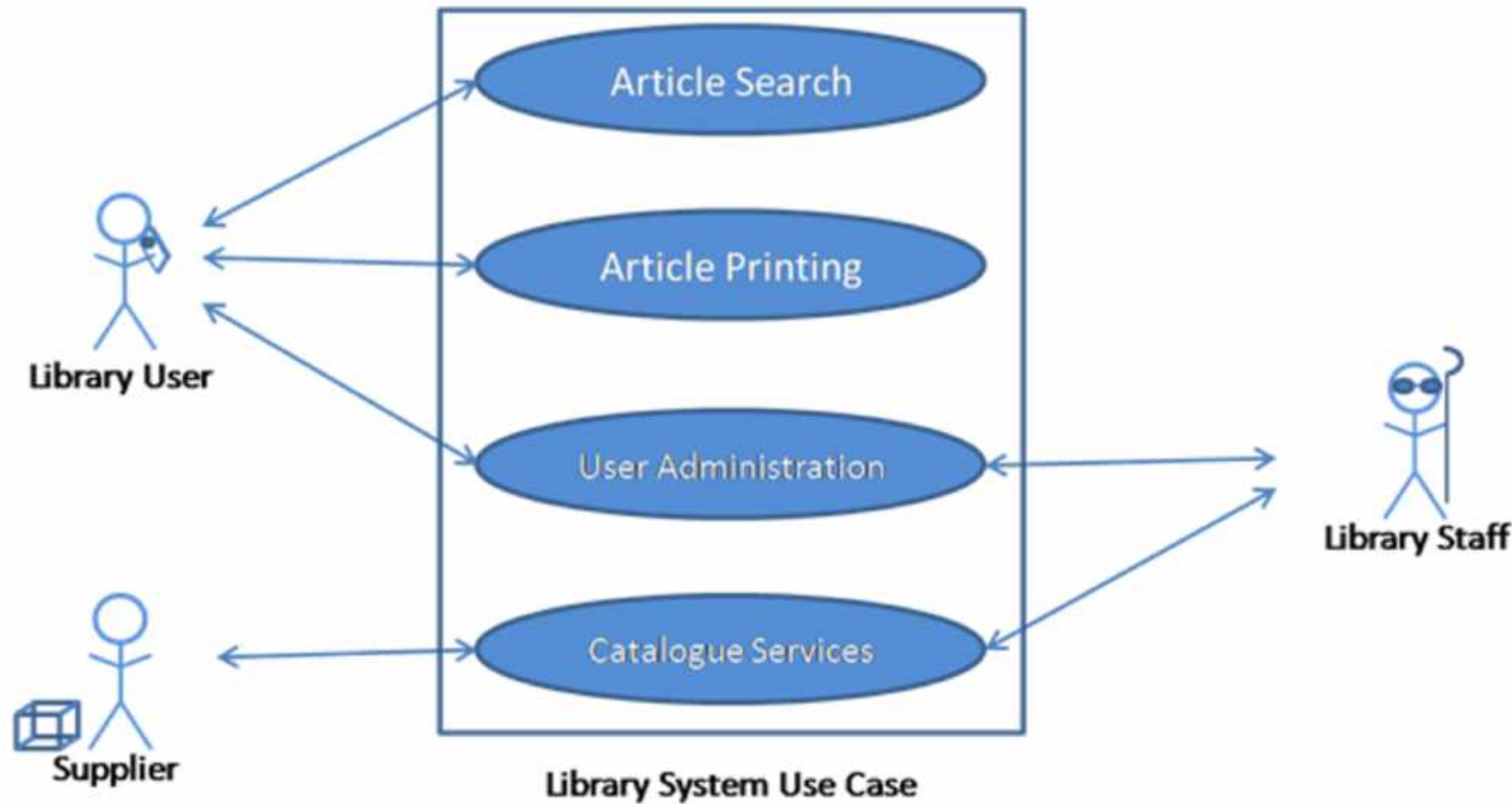


Condition	Action
Sugar level falling ($r_2 < r_1$)	CompDose = 0
Sugar level stable ($r_2 = r_1$)	CompDose = 0
Sugar level increasing and rate of increase decreasing ($(r_2 - r_1) < (r_1 - r_0)$)	CompDose = 0
Sugar level increasing and rate of increase stable or increasing $r_2 > r_1$ & $(r_2 - r_1) \geq (r_1 - r_0)$	CompDose = round $((r_2 - r_1)/4)$ If rounded result = 0 then CompDose = MinimumDose



Use Cases

Use Case



- A use case describes how a user (or another system) interacts with a system to achieve a goal.

- Use Cases are a scenario Based techniques in the UML which describe the interaction itself

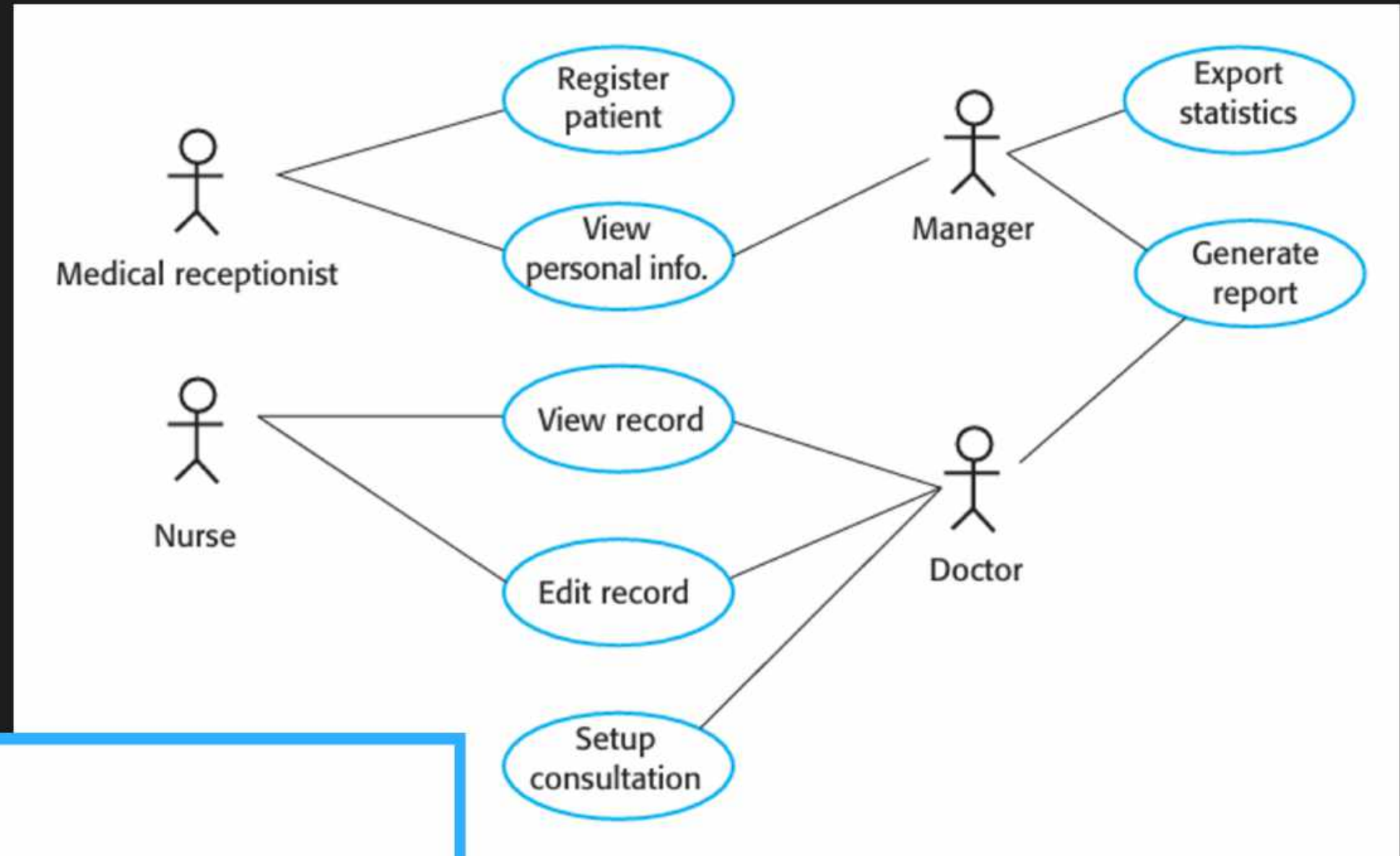
- Use cases are part of UML (Unified Modeling Language) and are commonly used in software engineering.

- Sequence Diagrams may be used to add detail to Use-cases by showing the sequence of the event processing in the system.

Actors in the System

- **Medical Receptionist**
- **Nurse**
- **Doctor**
- **Manager**

Each actor interacts with the system in different ways.



Use Case 1: Register Patient

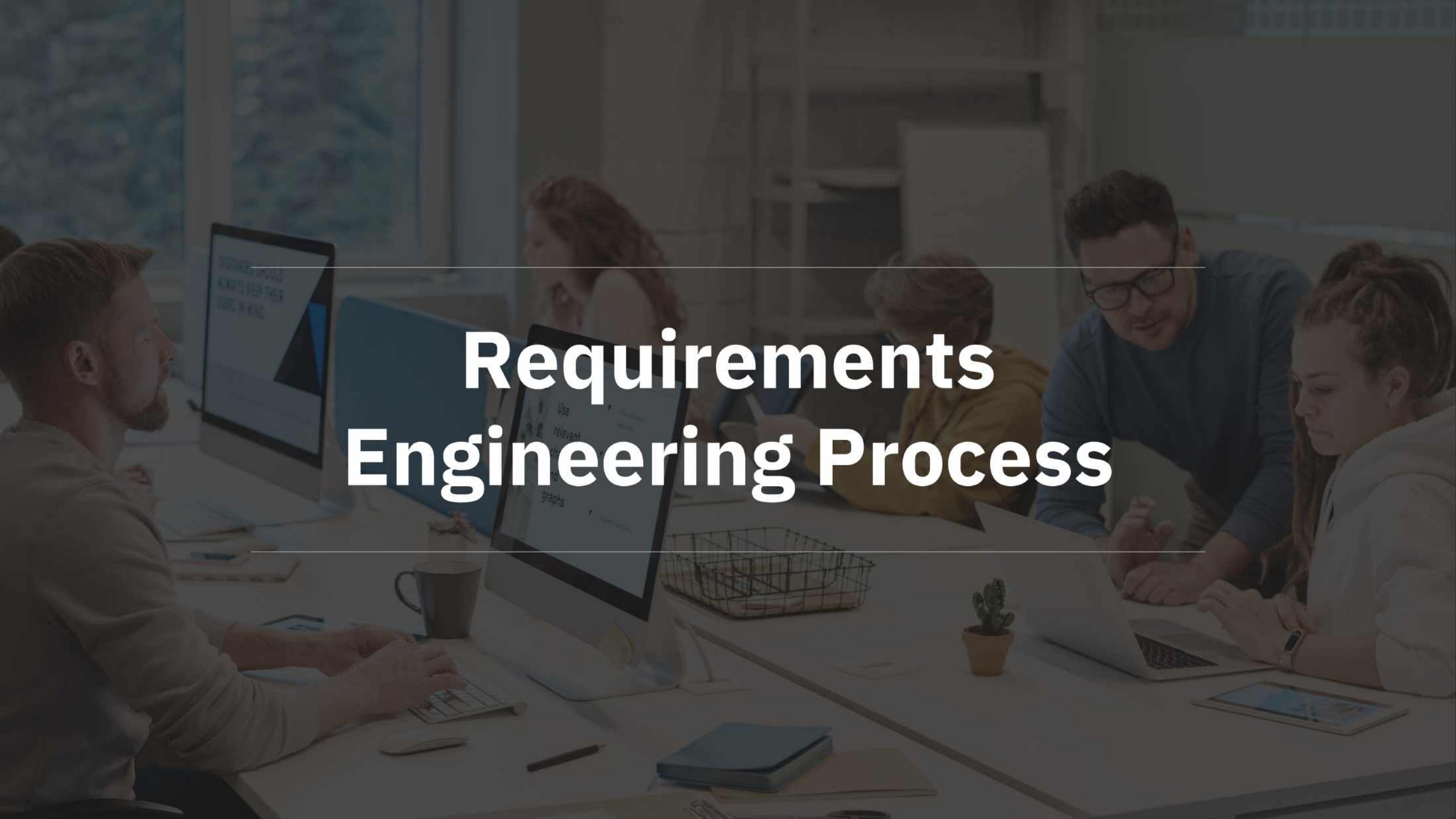
Actor: Medical Receptionist

Description:

The medical receptionist registers a new patient in the system. The receptionist enters personal details such as name, address, date of birth, and contact information. Once the information is validated, the system creates a new patient record and stores it securely.

Outcome:

A new patient record is created and available for medical staff.

A group of people are working together in a modern office environment. In the foreground, a man with a beard is seated at a desk, looking at a computer monitor. To his right, another man with glasses is leaning over a desk, looking at a laptop. A woman is seated next to him, also looking at the laptop. In the background, another woman is visible, working at a desk. The office has large windows, and the overall atmosphere is professional and collaborative. The text "Requirements Engineering Process" is overlaid in the center of the image.

Requirements Engineering Process

Requirement Engineering Process

The processes used for RE vary widely depending on the application domain, the people involved and the organisation developing the requirements.



Requirements elicitation and analysis

Discovering requirements by interacting with stakeholders



Requirements specification

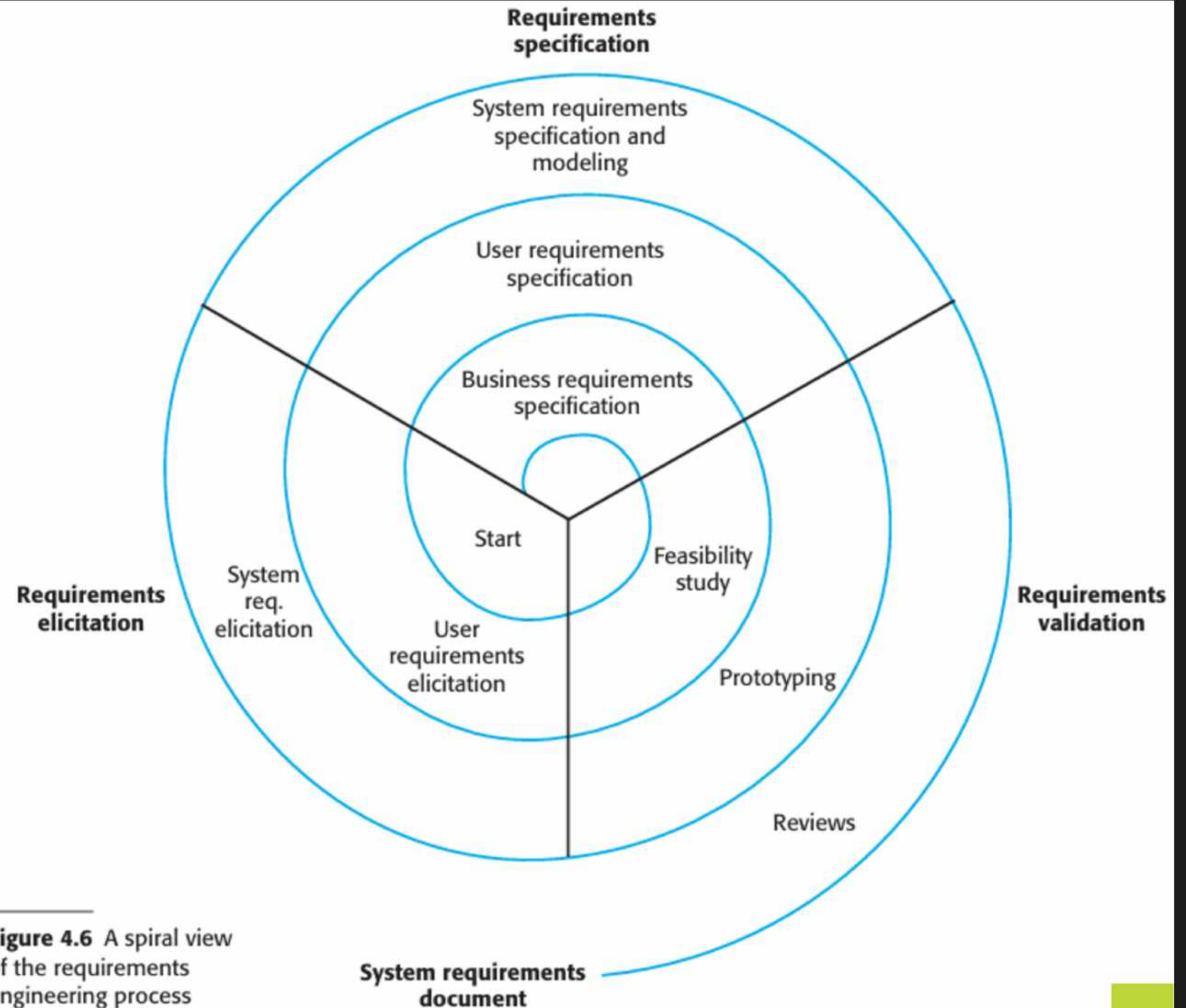
Converting these requirements into a standard form



Requirements validation

Checking that the requirements actually define the system that the customer wants

A spiral view of the requirements engineering process





Requirements elicitation and analysis

- Involves technical staff working with customers to find out about the application domain, the services that the system should provide and the system's operational constraints.

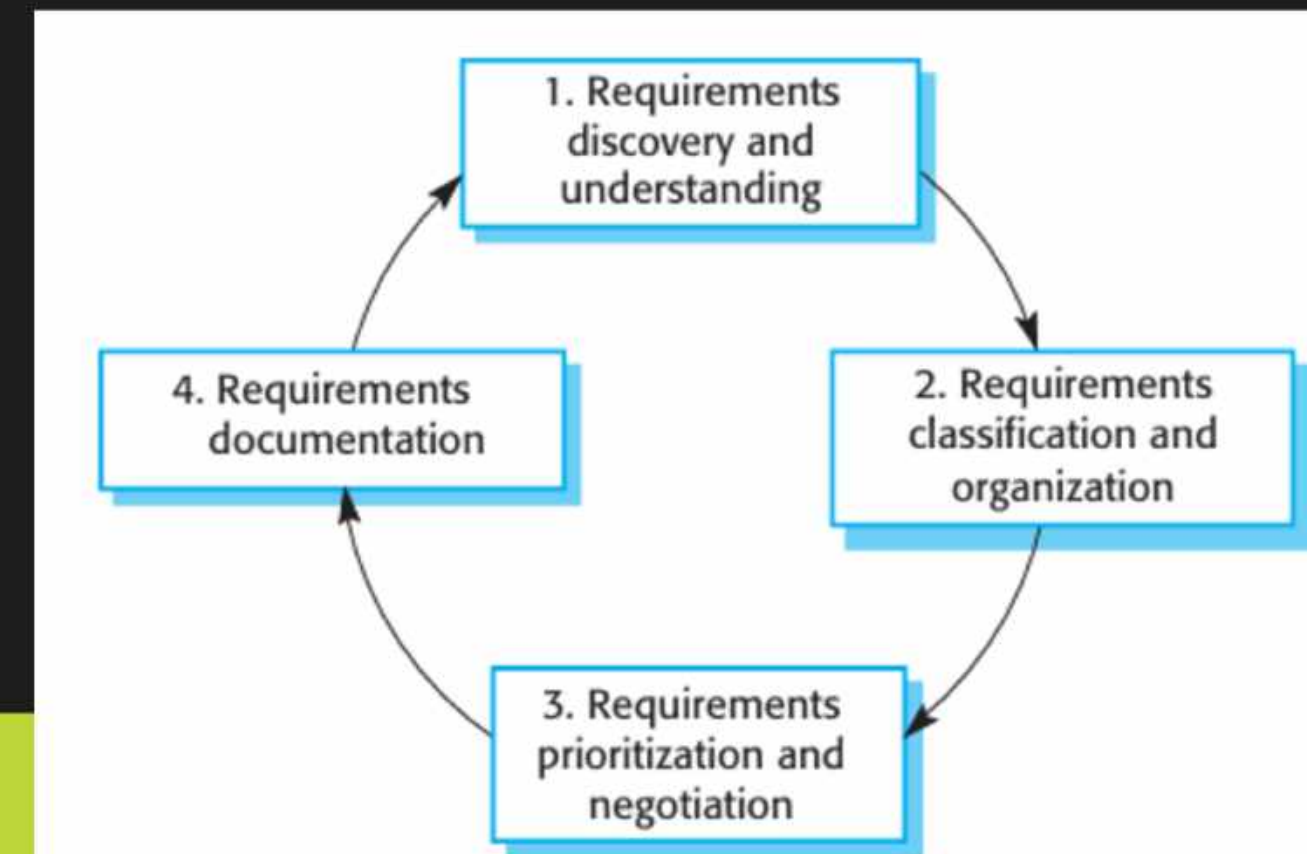
- May involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc. These are called stakeholders

- Problems:

- Stakeholders don't know what they really want.
- Stakeholders express requirements in their own terms
- Different stakeholders may have conflicting requirements
- The requirements change during the analysis process. New stakeholders may emerge and the business environment may change

- Stages include:

- Requirements discovery,
- Requirements classification and organization,
- Requirements prioritization and negotiation,
- Requirements specification.



A group of people are gathered in a meeting room, looking at a whiteboard. The room has a white wall with a framed picture on the left and a window with curtains in the background. The people are dressed in casual business attire. The text "Requirements Elicitation Techniques" is overlaid in the center of the image.

Requirements Elicitation Techniques

Two fundamental elicitation approaches

Interviewing



Observation or Ethnography



A group of people are gathered around a table in a meeting. In the foreground, a person is using a laptop that displays various charts and graphs. Other people are visible in the background, some looking at the laptop screen. The scene is dimly lit, with a dark overlay on the image.

Requirements Elicitation Techniques

Interviewing

- In these interviews, the requirements engineering team puts questions to stakeholders about the system that they currently use and the system to be developed.

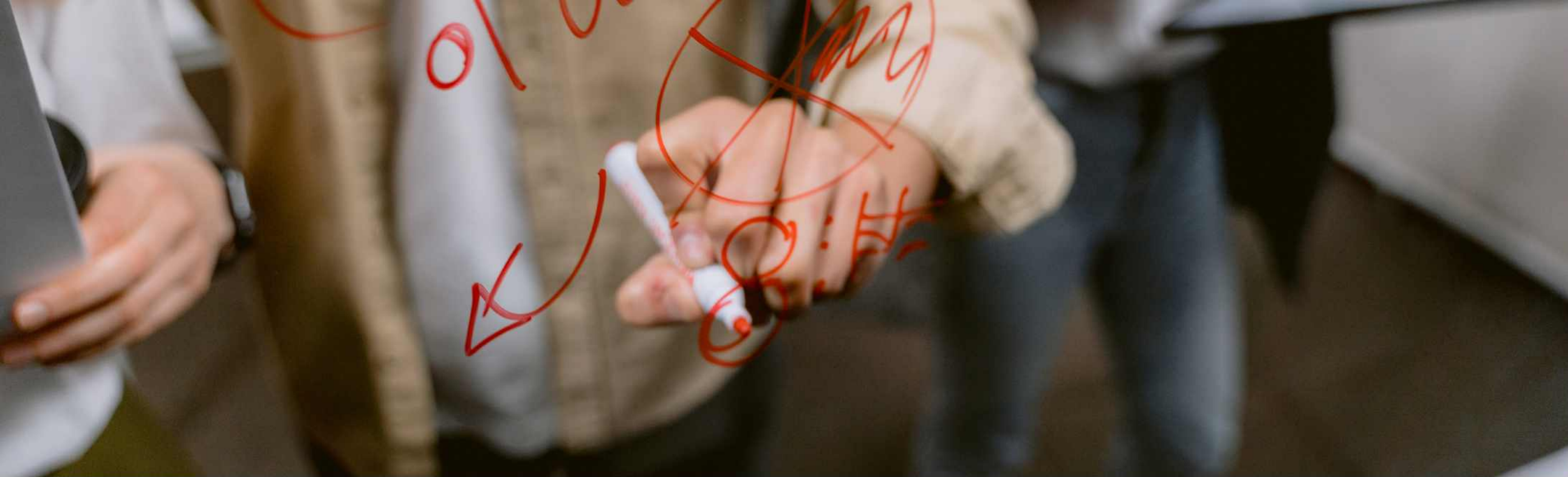
- Interviews may be of two types:

1. **Closed interviews**, where the stakeholder answers a predefined set of questions.
2. **Open interviews**, in which there is no predefined agenda. The requirements engineering team explores a range of issues with system stakeholders and hence develops a better understanding of their needs.

- Interviews are good for getting an overall understanding of what stakeholders do, how they might interact with the new system, and the difficulties that they face with current systems. However, unless you have a system prototype to demonstrate, you should not expect stakeholders to suggest specific and detailed requirements. Everyone finds it difficult to visualize what a system might be like.

- Problems in Eliciting Domain Knowledge:

- Use of domain-specific jargon
- Implicit knowledge not mentioned because it seems obvious

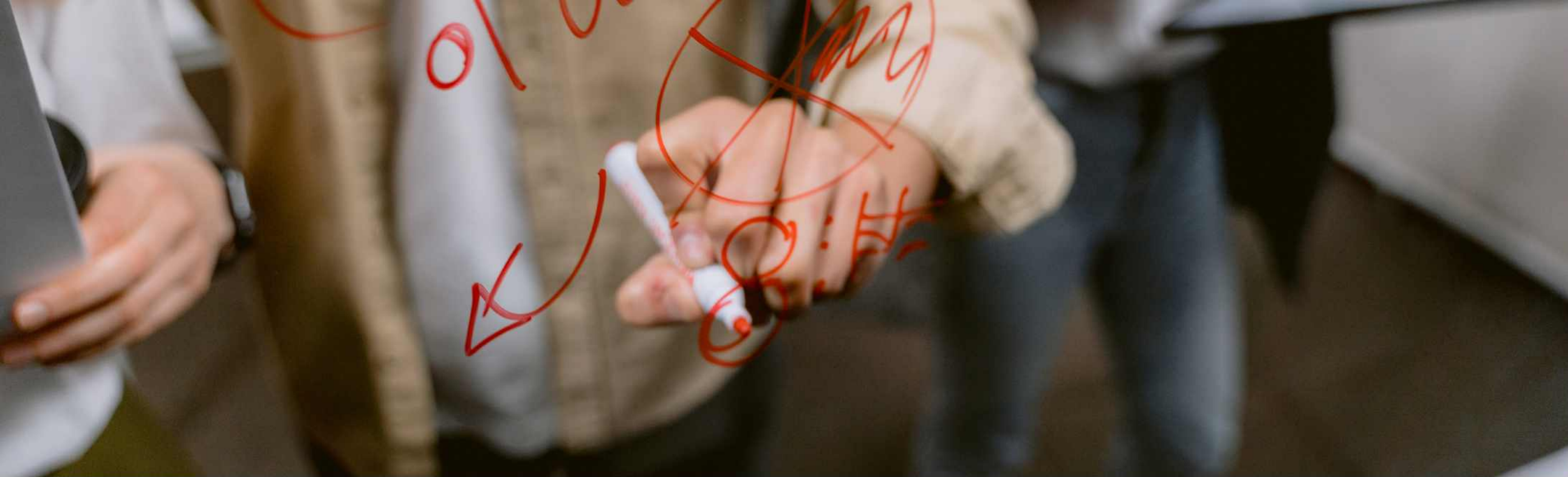


Scenarios

Scenarios are real-life examples of how a system can be used.

They should include:

- A description of the starting situation;
- A description of the normal flow of events;
- A description of what can go wrong;
- Information about other concurrent activities;
- A description of the state when the scenario finishes



Scenarios

Scenarios are real-life examples of how a system can be used.

User Requirement Scenario: **Borrowing a Book**

- **Start:** User is logged in; book is available
- **Action:** User selects a book and clicks Borrow
- **System response:** System records the loan and confirms
- **Error:** If unavailable, system shows an error message
- **End:** Book is assigned to user with a due date

A group of people are gathered around a table in a meeting. In the foreground, a person is using a laptop that displays various charts and graphs. Other people are visible in the background, some looking at the laptop screen. The scene is dimly lit, with a dark overlay on the image.

Requirements Elicitation Techniques

Ethnography

- One reason why many software systems are delivered but never used is that their requirements do not take proper account of how **social and organizational factors affect the practical operation of the system**

- Ethnography is an observational technique that can be used to ***understand operational processes*** and help derive requirements for software to support these processes. An analyst immerses himself or herself in ***the working environment where the system will be used***. The day-to-day work is observed, and notes are made of the actual tasks in which participants are involved. The value of ethnography is that it ***helps discover implicit system requirements*** that reflect the actual ways that people work, rather than the formal processes defined by the organization.

- Social and organizational factors that affect the work, but that are not obvious to individuals, may only become clear when noticed by an unbiased observer. For example, a workgroup may self-organize so that members know of each other's work and can cover for each other if someone is absent. This may not be mentioned during an interview as the group might not see it as an integral part of their work.

Ethnography is particularly effective for discovering two types of requirements:



Real work vs. documented processes

Requirements derived from the way in which people actually work, rather than the way in which business process definitions say they ought to work. In practice, people never follow formal processes.

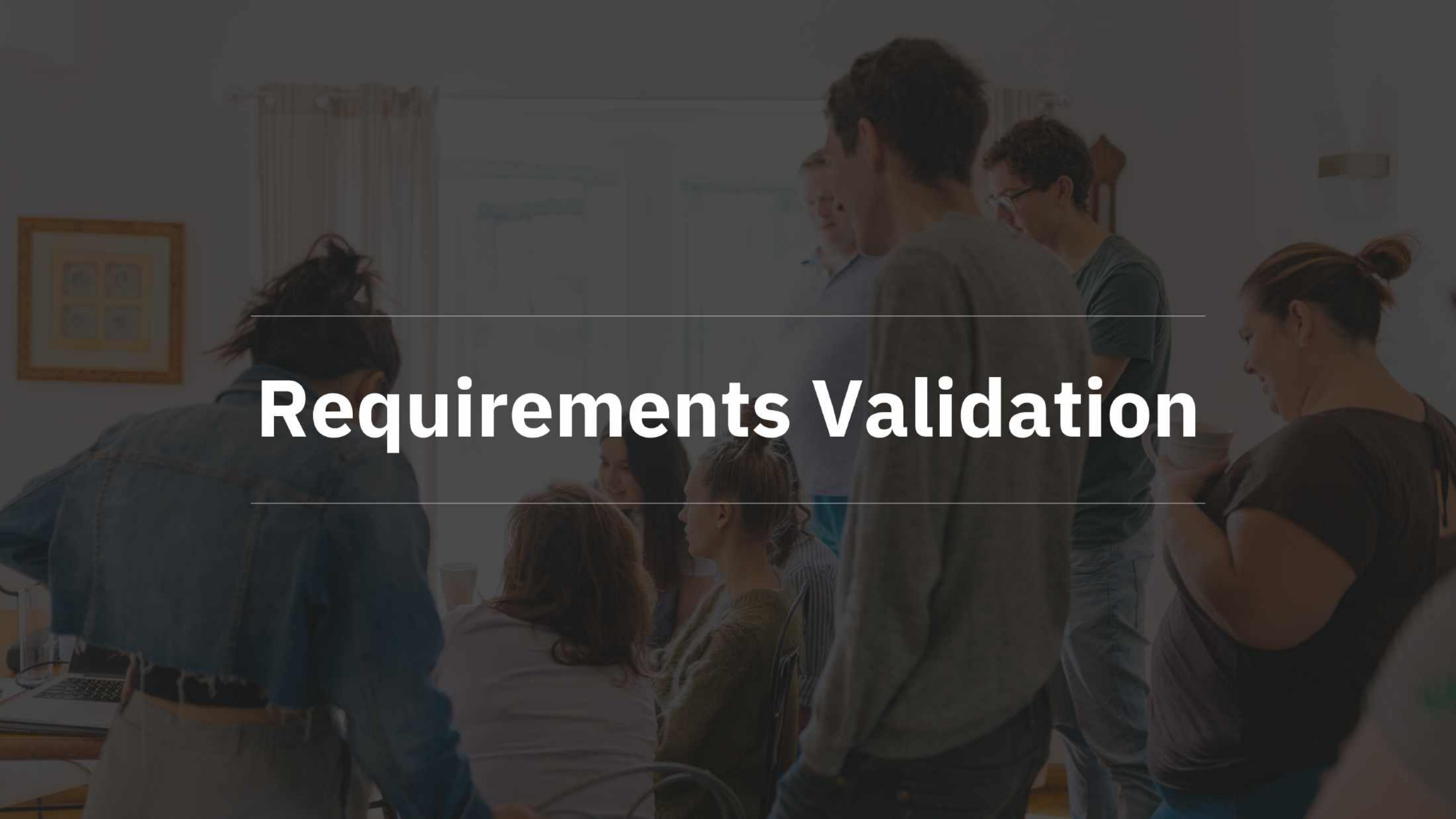
For example, air traffic controllers may switch off a conflict alert system that detects aircraft with intersecting flight paths, even though normal control procedures specify that it should be used. The conflict alert system is sensitive and issues audible warnings even when planes are far apart. Controllers may find these distracting and prefer to use other strategies to ensure that planes are not on conflicting flight paths.



Cooperation and situational awareness

Requirements derived from cooperation and awareness of other people's activities.

For example, air traffic controllers (ATCs) may use an awareness of other controllers' work to predict the number of aircraft that will be entering their control sector. They then modify their control strategies depending on that predicted workload. Therefore, an automated ATC system should allow controllers in a sector to have some visibility of the work in adjacent sectors.

A group of people are gathered in a meeting room, looking at a whiteboard. The scene is dimly lit, with the whiteboard being the primary light source. The people are dressed in casual business attire. The whiteboard contains some diagrams and text, but it is not clearly legible. The overall atmosphere is professional and collaborative.

Requirements Validation



Requirements Validation

Requirements validation is the process of checking that requirements define the system that the customer really wants.

- Requirements validation is **critically important because errors in a requirements document can lead to extensive rework costs** when these problems are discovered during development or after the system is in service.
- The **cost of fixing** a requirements problem by making a system change is **usually much greater than repairing design or coding errors**. A change to the requirements usually means that the system design and implementation must also be changed.

During the requirements validation process, different types of checks should be carried out on the requirements in the requirements document. These checks include:

- 1. **Validity checks** These check that the requirements reflect the real needs of system users. Because of changing circumstances, the user requirements may have changed since they were originally elicited.
- 2. **Consistency checks** Requirements in the document should not conflict. That is, there should not be contradictory constraints or different descriptions of the same system function.
- 3. **Completeness checks** The requirements document should include requirements that define all functions and the constraints intended by the system user.
- 4. **Realism checks** By using knowledge of existing technologies, the requirements should be checked to ensure that they can be implemented within the proposed budget for the system. These checks should also take account of the budget and schedule for the system development.
- 5. **Verifiability** To reduce the potential for dispute between customer and contractor, system requirements should always be written so that they are verifiable. This means that you should be able to write a set of tests that can demonstrate that the delivered system meets each specified requirement.

A number of requirements validation techniques can be used individually or in conjunction with one another:

- 1. **Requirements reviews** The requirements are analyzed systematically by a team of reviewers who check for errors and inconsistencies.
- 2. **Prototyping** This involves developing an executable model of a system and using this with end-users and customers to see if it meets their needs and expectations. Stakeholders experiment with the system and feed back requirements changes to the development team.
- 3. **Test-case generation** Requirements should be testable. If the tests for the requirements are devised as part of the validation process, this often reveals requirements problems. If a test is difficult or impossible to design, this usually means that the requirements will be difficult to implement and should be reconsidered. Developing tests from the user requirements before any code is written is an integral part of test-driven development

Requirements review checks

Verifiability: Is the requirement realistically testable?

Comprehensibility: Is the requirement properly understood?

Traceability: Is the origin of the requirement clearly stated?

Adaptability: Can the requirement be changed without a large impact on other requirements?



A group of people are gathered in a meeting room, looking at a whiteboard. The scene is dimly lit, with the whiteboard being the primary light source. The people are dressed in casual business attire. A woman in the foreground is wearing a denim jacket. A man in the center is wearing a grey sweater. A woman on the right is holding a mug. The whiteboard has some diagrams and text on it, but it's not clearly legible. The overall atmosphere is professional and collaborative.

Requirements Management

- **Requirements management** is the process of managing changing requirements during the requirements engineering process and system development.
- **New requirements emerge** as a system is being developed and after it has gone into use.
- You need to **keep track of individual requirements and maintain links between dependent requirements** so that you can assess the impact of requirements changes. You need to establish a formal process for making change proposals and linking these to system requirements.

Changing Requirements

The business and technical environment of the system always changes after installation.

New hardware may be introduced, it may be necessary to interface the system with other systems, business priorities may change (with consequent changes in the system support required), and new legislation and regulations may be introduced that the system must necessarily abide by.



The people who pay for a system and the users of that system are rarely the same people

System customers impose requirements because of organizational and budgetary constraints. These may conflict with end-user requirements and, after delivery, new features may have to be added for user support if the system is to meet its goals

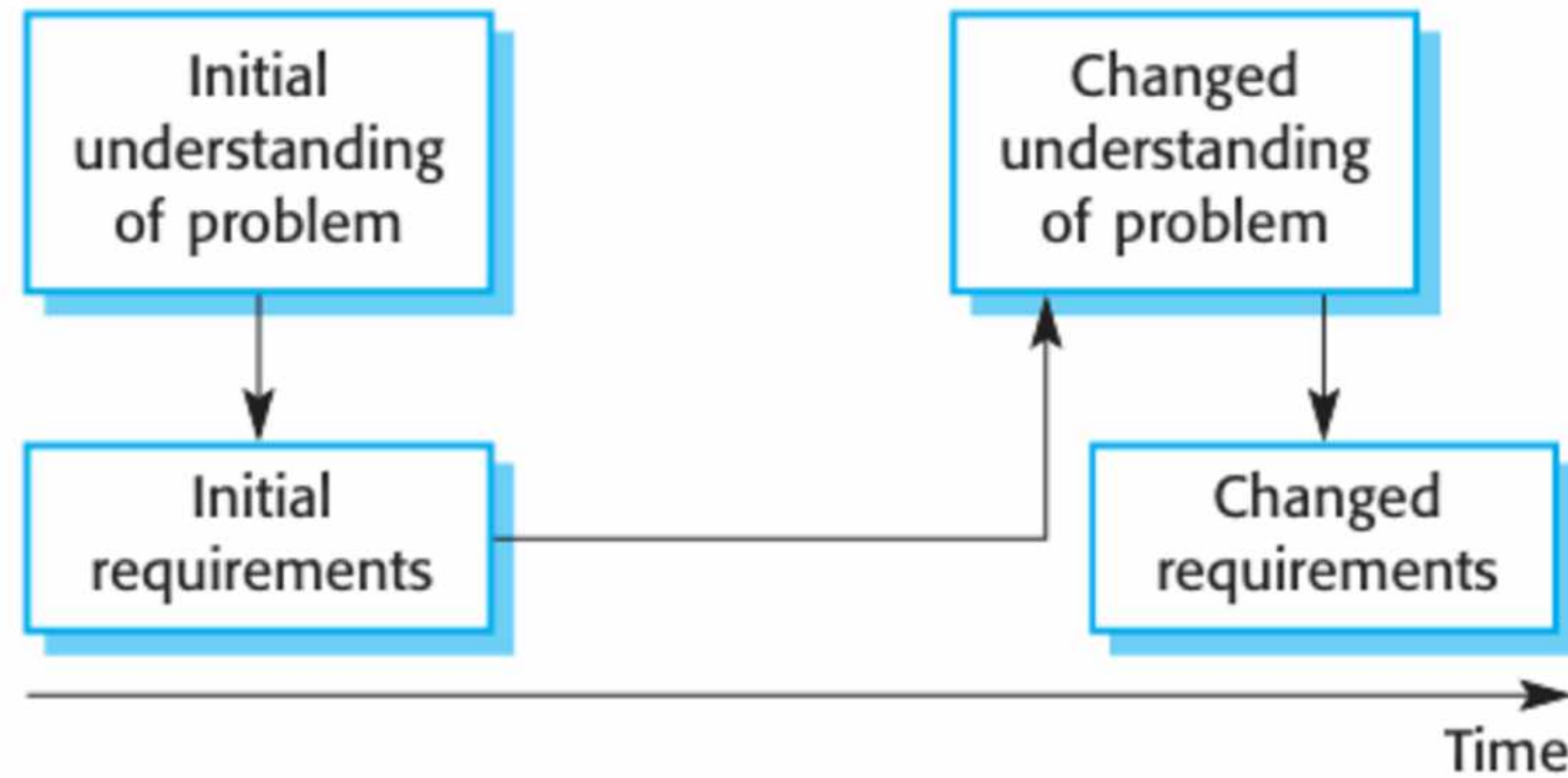


Large systems usually have a diverse user community, with many users having different requirements and priorities that may be conflicting or contradictory.

Final system requirements are inevitably a compromise between them and, with experience, it is often discovered that the balance of support given to different users has to be changed.



Figure 4.18
Requirements evolution



Requirements Evolution

A group of people are working in an office environment. In the foreground, a man with a beard is seated at a desk, looking at a computer monitor. To his right, another man with glasses is leaning over a desk, looking at a laptop. A woman is seated next to him, also looking at the laptop. In the background, another woman is visible, working at a desk. The office has large windows in the background, and the overall atmosphere is professional and collaborative. The text 'Requirements Management Planning' is overlaid in the center of the image in a large, white, sans-serif font.

Requirements Management Planning

Requirements management planning is concerned with establishing how a set of evolving requirements will be managed.

Requirements identification

Each requirement must be uniquely identified so that it can be cross-referenced with other requirements and used in traceability assessments.

A change management process

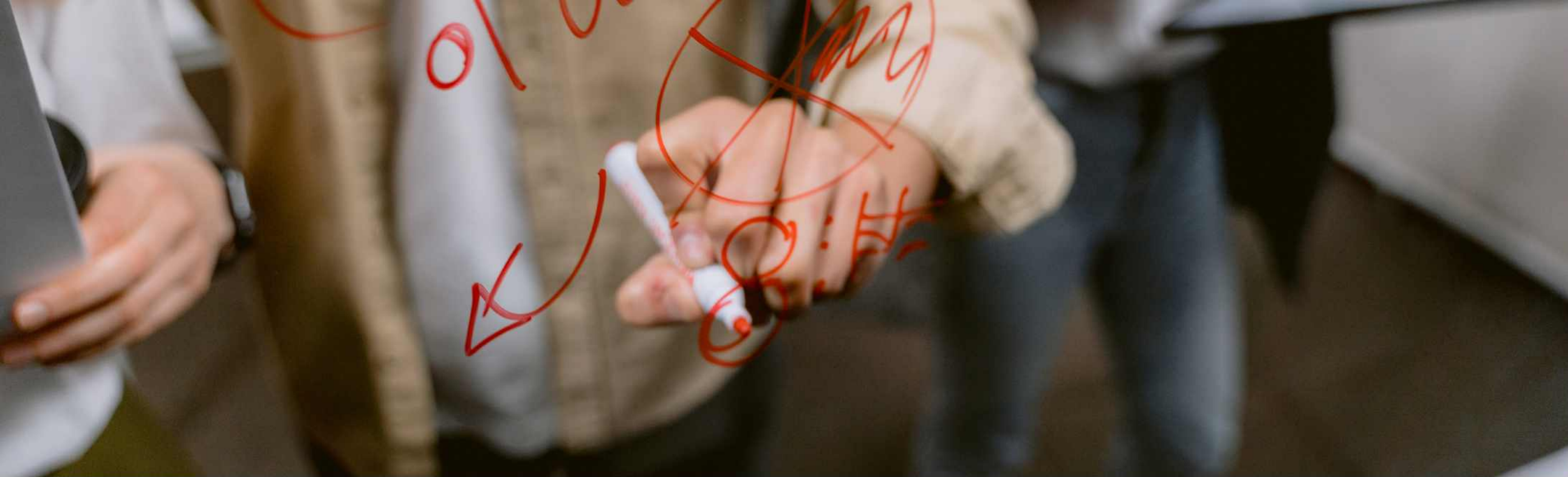
This is the set of activities that assess the impact and cost of changes.

Traceability policies

These policies define the relationships between each requirement and between the requirements and the system design that should be recorded. The traceability policy should also define how these records should be maintained.

Tool support

Requirements management involves the processing of large amounts of information about the requirements. Tools that may be used range from specialist requirements management systems to shared spreadsheets and simple database systems.



Requirements change management

Change management is essential because you need to decide if the benefits of implementing new requirements are justified by the costs of implementation

- **There are three principal stages to a change management process:**

Problem analysis and change specification: A problem is identified in an existing requirement or a new change request is proposed by a stakeholder

- The request is analyzed to determine:
 - Is the problem real?
 - Is the change valid and necessary?
 - Is the request clear and well-defined?

The results are fed back to the change requestor.

Possible outcomes:

- The requestor:
 - Refines the request with more detail
 - Withdraws the request after understanding implications

There are three principal stages to a change management process:

- **Change analysis and costing:** The impact of the proposed change is analyzed

Traceability information is used to identify:

- Affected requirements
- Design components
- Code modules
- Test cases

The cost of the change is estimated in terms of:

- Time
- Effort
- Risk
- Budget

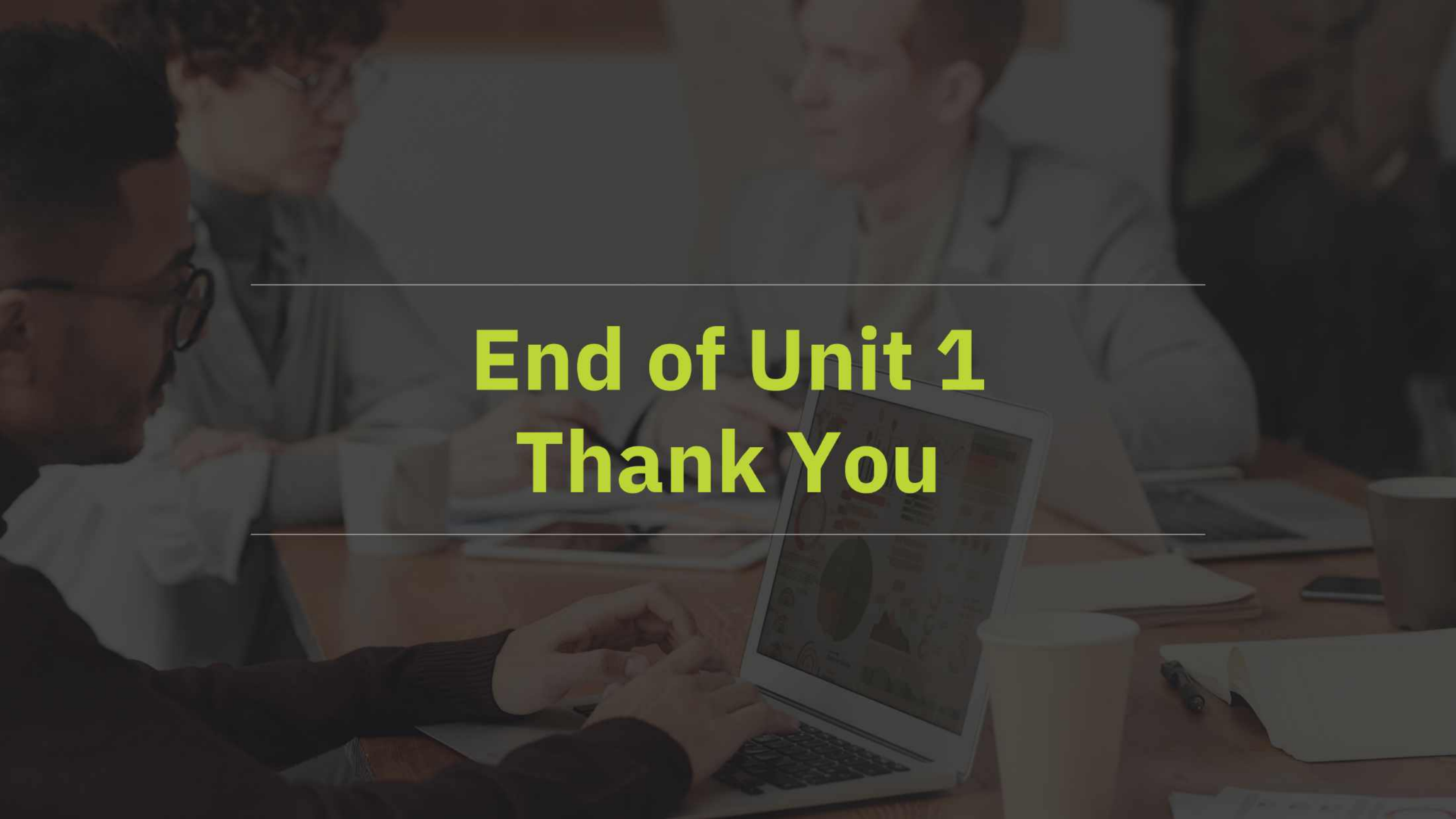
Based on this, a decision is made to:

- Approve the change
- Defer the change
- Reject the change

- **There are three principal stages to a change management process:**

- **Change implementation:** Approved changes are implemented
- Updates are made to:
 - Requirements document
 - System design documents
 - Software implementation
 - Test cases

The requirements document should be structured to allow easy modification.

A group of people are gathered around a table in a meeting. In the foreground, a person's hands are visible, typing on a laptop. The laptop screen displays several charts, including a pie chart and a bar chart. Other people are visible in the background, some looking at the laptop and others looking away. The image has a dark, semi-transparent overlay.

End of Unit 1 Thank You